

DSM500 Final Project Report

Synthetic Markets, Real Decisions: Evaluating the Downstream Utility of Generative Models for Trading-Strategy Validation

Author: Guillermo Enrique Olmos Ranalli (200153474)

Module: DSM500 - Data Science MSc Final Project

Date: October 2026

Contents

- [Abstract](#)
- [1. Introduction](#)
 - [1.1 Motivation: The Backtesting Problem](#)
 - [1.2 A Natural Solution: Synthetic Market Data](#)
 - [1.3 The Critical Gap: Statistical Realism \$\neq\$ Task Relevance](#)
 - [1.4 Aim and Objectives](#)
 - [1.5 Research Questions](#)
 - [1.6 Stakeholders](#)
 - [1.7 Preview of Findings](#)
- [2. Literature Review](#)
 - [2.1 Backtest Overfitting and Data Snooping](#)
 - [2.2 Stylized Facts as the Yardstick for Realism](#)
 - [2.3 Classical Generators: Resampling and Volatility Models](#)
 - [2.4 Deep Market Generators](#)
 - [2.5 How Synthetic Financial Data Are Evaluated](#)
 - [2.6 Gap and Positioning](#)
- [3. Methods and Experimental Design](#)
 - [3.1 Data](#)
 - [3.2 Windowing Strategy](#)
 - [3.3 Generators Evaluated](#)
 - [3.4 Evaluation Metrics](#)
 - [3.5 Trading Rules Universe](#)
 - [3.6 Testing and Verification](#)
 - [3.7 Reproducibility](#)
 - [3.8 Implementation](#)
- [4. Results](#)
 - [4.1 RQ1: Stylized Facts Fidelity](#)
 - [4.2 RQ2: Downstream Task-Based Utility \(Primary Result\)](#)
 - [4.3 RQ2 Secondary Metrics: False Discovery, Optimism and Selection Regret](#)
 - [4.4 Robustness and Paired Comparison of Generators](#)
 - [4.5 Sensitivity by Rule Family](#)
 - [4.6 Summary of Results](#)
- [5. Discussion](#)
 - [5.1 Why Might GARCH Rank Strategies Better than the Bootstrap?](#)
 - [5.2 Why the Deep Generators Underperform](#)
 - [5.3 Relation to Prior Work](#)
 - [5.4 Limitations](#)
 - [5.5 Changes from the Proposal](#)
 - [5.6 Reflection](#)
- [6. Conclusions](#)

- 6.1 Answering the Research Questions
- 6.2 Contribution
- 6.3 Practical Implications
- 6.4 Future Work
- 7. Ethical Considerations
- 8. References
- 9. Appendices
 - A. Data Summary
 - B. GAN Training
 - C. Rule Universe
 - D. Code Availability
 - E. Code Listings

Abstract

This project evaluates generative models of financial returns by their downstream usefulness for strategy validation, rather than by statistical realism alone. A universe of technical trading rules was backtested on held-out USD/EUR data (era-B: 2022–2024) and on 1,000 synthetic paths from each generator, each trained or parameterised using era-A (2018–2021).

Usefulness is measured by the Spearman rank correlation between each rule's synthetic and real Sharpe ratio. After merging rule specifications that the pipeline treated as the same rule, 185 distinct rules remain (181 for the two GAN generators).

Main finding. GARCH(1,1) gave the strongest rank agreement ($\rho = 0.54$, 95% CI 0.42–0.63), ahead of a window bootstrap ($\rho = 0.34$, 0.20–0.46), an LSTM GAN ($\rho = 0.30$, 0.16–0.43) and an MLP GAN ($\rho = 0.19$, 0.05–0.33). All four correlations are significantly above zero. A paired resampling comparison on the 181 rules common to all generators puts GARCH ahead of each of the others (differences of 0.21 to 0.32, with every 95% interval above zero), while the bootstrap, LSTM GAN and MLP GAN cannot be separated from one another. Because many rules are close variants, these intervals are probably too narrow. Moving-average crossover rules make up 180 of the 185 distinct rules, so the finding applies to that family.

Interpretation. The window bootstrap reproduces the training data's moments almost exactly, yet ranks strategies less well than GARCH. This is consistent with, though it does not prove, the view that resemblance metrics alone are an insufficient test of synthetic financial data (Assefa et al., 2019).

Caveats. The backtester computes trade returns incorrectly, GARCH uses literature-default rather than fitted parameters, a genuine walk-forward baseline was not run, and the comparison covers one asset and one test period (Section 5.4). The results are best read as a pilot of the evaluation framework rather than a definitive ranking of generators.

1. Introduction

1.1 Motivation: The Backtesting Problem

Backtesting—evaluating trading strategies on historical data before deployment—is the central tool in quantitative finance. Yet it is notoriously unreliable. Because strategies are tuned on a single realised price path, strong in-sample performance frequently fails to persist out-of-sample, a manifestation of overfitting and data-snooping bias documented extensively in the literature (White, 2000; Bailey et al., 2017; Harvey, Liu and Zhu, 2016).

The consequences are severe:

- Capital is deployed to strategies whose apparent edge is an artifact of noise
- The published record of "profitable" strategies is inflated by selection bias from configurations silently tested and discarded
- Harvey, Liu and Zhu (2016) argue that, once the scale of searching is accounted for, many published findings about expected returns are likely to be false discoveries

1.2 A Natural Solution: Synthetic Market Data

If we could generate many plausible alternative price paths—each realistic in its statistical properties but different from history—we could validate whether strategy performance is robust across them. Recent advances in deep generative models (GANs, diffusion models) make this possible. A growing body of work shows such "market generators" can reproduce salient stylized facts of real returns: heavy tails, volatility clustering, near-zero autocorrelation (Wiese et al., 2020; Takahashi, Chen and Tanaka-Ishii, 2019).

1.3 The Critical Gap: Statistical Realism $\neq$ Task Relevance

However, prior work evaluates generators mainly on statistical fidelity—how closely synthetic data matches moment distributions, autocorrelations, and other stylized facts. This evaluation approach has a blind spot: **a generator can match every statistical property and still be useless (or actively misleading) for a downstream decision, because realism in aggregate statistics does not guarantee that the specific temporal dependence a strategy exploits is preserved.**

Assefa et al. (2019) note that there is still no agreed way to evaluate synthetic financial data, and that statistical-distance measures cannot be relied on as evidence of realism. This project takes a complementary, task-based route: we compare generators not just on how realistic they are, but on whether strategies validated on synthetic data actually predict real out-of-sample performance—the decision that matters to practitioners.

1.4 Aim and Objectives

Aim. To develop and evaluate a task-based framework that measures generative market models by the quality of the strategy-validation decisions they support, and to establish whether deep generators offer more validation utility than classical resampling and volatility-modelling baselines.

Objectives.

1. Implement deep generative models of daily return windows (an MLP GAN and an LSTM GAN) alongside two classical baselines (GARCH(1,1) and a window bootstrap).
2. Evaluate each generator's statistical fidelity with stylized-fact diagnostics (RQ1).
3. Build a task-based protocol that backtests a universe of trading rules on each data source and measures how well the resulting ranking predicts held-out real performance, with complementary decision metrics (RQ2).
4. As an extension, examine how this utility varies across assets and market regimes (RQ3).
5. Deliver the pipeline in a reproducible, tested form.

Section 5.5 records where the delivered study departs from these objectives.

1.5 Research Questions

The project is organised around three comparative research questions:

- **RQ1:** How faithfully do deep generative models reproduce the stylized facts of real financial time series, relative to GARCH and bootstrap baselines?
 - *Rationale:* Establishes that statistical fidelity is achievable; baseline for comparison to RQ2.
- **RQ2 (Primary):** To what extent does validating trading strategies on generated data improve the accuracy of out-of-sample performance estimates and reduce false discovery, compared with single-history walk-forward validation and classical resampling?
 - *Rationale:* Measures downstream decision quality—the core innovation. Does synthetic validation actually help?
- **RQ3 (Exploratory):** How does the downstream utility of each validation method vary across asset classes and market regimes?
 - *Rationale:* Explores generalisation; deferred to future work if time-constrained.

1.6 Stakeholders

Several groups stand to benefit from a reliable, task-based way to validate trading strategies.

Quantitative researchers and strategy developers gain a check on whether an apparently profitable backtest is likely to survive out of sample. **Risk and investment committees**, who decide whether to allocate capital, gain an interpretable measure of how far a validation

method can be trusted and how much it inflates expectations. **Practitioners weighing tooling costs** gain evidence on whether expensive deep generators are worth their cost or whether cheap baselines suffice. The **academic and open-source community** gains a reusable framework that judges market generators by decision usefulness rather than statistical realism alone. The project makes no investment recommendation; its value lies in better-informed validation, not in trading signals.

1.7 Preview of Findings

We compare four generators (parametric GARCH(1,1), a nonparametric window bootstrap, an MLP GAN and an LSTM GAN) on their ability to predict which trading rules perform well out-of-sample. Each rule is backtested on the 526 real era-B USD/EUR windows and on 1,000 synthetic paths per generator, and we measure the Spearman rank correlation between the two sets of Sharpe ratios.

RQ1. The window bootstrap reproduces the training data's moments almost exactly (mean autocorrelation over lags 1–20 -0.00177 real vs -0.00178 synthetic; kurtosis 22.43 vs 21.99). The pipeline did not produce fidelity statistics for GARCH or the GANs, so RQ1 is answered for the bootstrap only.

RQ2. On 185 distinct rules, GARCH achieves $p = 0.54$, the window bootstrap 0.34, the LSTM GAN 0.30 and the MLP GAN 0.19. In paired comparisons GARCH is ahead of each of the other three, which cannot be separated from one another.

Methodological point. The generator that matches moment statistics most closely is not the one that ranks strategies best, which supports evaluating generators on the downstream task as well as on fidelity.

2. Literature Review

This review brings together four strands of work: the statistics of backtest overfitting, the stylized facts used to judge market realism, the generators that produce synthetic markets, and the way those generators are evaluated. The aim is to show where they meet and where they leave a gap.

2.1 Backtest Overfitting and Data Snooping

That backtests overstate future performance is well established. White (2000) frames the problem as data snooping: when many rules are tried on the same data, the best one looks good partly by chance, and his Reality Check uses a bootstrap to test whether the best rule beats a benchmark once the search is accounted for. Harvey, Liu and Zhu (2016) make the same argument for the cross-section of expected returns. Given the hundreds of factors already tested, they argue, the conventional significance threshold is far too lenient and a much higher hurdle is needed. Bailey and López de Prado (2014) translate the idea into a performance measure, the Deflated Sharpe Ratio, which discounts an observed Sharpe ratio for the number of trials and for non-normal returns. Bailey et al. (2017) estimate the Probability of Backtest Overfitting, the chance that the configuration chosen in-sample underperforms out of sample, using combinatorially symmetric cross-validation. López de Prado (2018) restructures validation itself, proposing purged and combinatorial cross-validation to limit leakage between training and test folds.

These methods differ in what they deliver: a significance test (White; Harvey, Liu and Zhu), an adjusted performance figure (the Deflated Sharpe Ratio), a probability (PBO) or a validation design (purged cross-validation). They share one feature: all of them work with the single observed history. They correct or discount a selection after it has been made, but cannot say how a rule would have behaved under different but plausible histories. Synthetic data offers that complementary route, by enlarging the set of histories on which rules are validated. It is only useful if the synthetic histories preserve whatever makes rules succeed or fail, which is the question this project tests.

2.2 Stylized Facts as the Yardstick for Realism

Cont (2001) catalogues the statistical regularities of asset returns: heavy tails, near-zero linear autocorrelation, volatility clustering (slowly decaying autocorrelation of absolute or squared returns), the leverage effect, gain/loss asymmetry and aggregational Gaussianity. These properties have become the standard yardstick for synthetic market data. A generator is considered realistic if its returns reproduce them.

For strategy validation the yardstick has two weaknesses. First, stylized facts are low-order summaries: a generator can match the distribution of returns and their autocorrelations while still misplacing the trend persistence or the timing of volatility bursts on which a trading rule's profit depends. Second, they are hard to turn into a single score. Assefa et al. (2019)

observe that stylized facts are often assessed qualitatively and had not yet been converted into a convincing single real-valued metric. Realism is therefore assessed piecemeal, and its link to downstream decisions is rarely tested.

2.3 Classical Generators: Resampling and Volatility Models

Two classical families are natural baselines. The stationary bootstrap of Politis and Romano (1994) resamples blocks of random length, which preserves short-range dependence while keeping the resampled series stationary. Resampling methods reproduce the empirical distribution exactly and need no model, but they cannot produce events outside the observed sample, and dependence longer than the blocks is lost. The window bootstrap used in this project is the extreme case: it resamples whole 256-day windows, so each synthetic path is a piece of real history.

The GARCH model of Bollerslev (1986) instead models the conditional variance recursively. A large shock raises tomorrow's variance, which produces volatility clustering and, even with Gaussian innovations, fat-tailed unconditional returns. The symmetric GARCH(1,1) does not capture the leverage effect, and with Gaussian innovations it usually produces less excess kurtosis than observed. Assefa et al. (2019) summarise the trade-off: autoregressive and GARCH-type models are easy to fit and interpret but rest on strong assumptions and miss many features of real data. Both families are cheap, transparent and well understood, which makes them the natural benchmark any more complex generator should be required to beat.

2.4 Deep Market Generators

Generative adversarial networks (Goodfellow et al., 2014) train a generator to fool a discriminator, and several studies adapt them to financial time series. Takahashi, Chen and Tanaka-Ishii (2019) show that a GAN can reproduce several stylized facts of financial returns. Wiese et al. (2020) introduce Quant GANs, which use temporal convolutional networks to capture long-range dependence such as volatility clustering. They evaluate the generated series with distributional distances and stylized-fact comparisons. Yoon, Jarrett and van der Schaar (2019) propose TimeGAN for general time series, adding a supervised step-ahead loss so that the model learns temporal dynamics rather than only the distribution of each time step. Notably, they assess generated data not only for fidelity but for usefulness, through a predictive score: a sequence model is trained on synthetic data and tested on real data. Denoising diffusion models (Ho, Jain and Abbeel, 2020) offer an alternative to adversarial training and have since been adapted to time series. Diffusion-TS (Yuan and Qiao, 2024) generates general multivariate series with an encoder-decoder transformer and an interpretable trend and seasonal decomposition. FTS-Diffusion (Huang, Chen and Qiao, 2024) targets financial series specifically. It extracts recurring patterns that vary in duration and magnitude, synthesises them with a diffusion network and models their succession; augmenting real data with its samples reduced stock-prediction error by up to 17.9% in the authors' experiments. Diffusion models were not implemented in this project (Section 5.5).

Two points stand out. First, progress is measured mainly by resemblance to real data. Where usefulness is tested, the downstream task is prediction: a forecasting model trained on, or augmented with, synthetic data (Yoon, Jarrett and van der Schaar, 2019; Huang, Chen and Qiao, 2024). None of these studies asks whether synthetic data lead to better selection of trading strategies. Second, deep generators are data-hungry and can be unstable to train, while daily financial series offer only a few thousand observations per asset. This tension matters for the results in Section 4.

2.5 How Synthetic Financial Data Are Evaluated

Assefa et al. (2019) survey synthetic data in finance, with a strong emphasis on privacy, and identify measuring the similarity between real and generated data as one of three open problems. They note that the field has no common benchmarks, datasets or metrics. Drawing on work in image generation, they argue that statistical-distance measures cannot be relied on as metrics of realism. They also observe that backtest-style evaluations, such as comparing a generator's Value-at-Risk with the value later observed, suffer from a dearth of data because only one history is available. Among the motivations for synthetic data they list the lack of historical data and the usefulness of counterfactual data for testing strategies.

Synthetic data have also been used directly for backtesting. López de Prado (2018, ch. 13) fits a discrete Ornstein-Uhlenbeck process to historical prices, generates many synthetic paths and uses them to choose profit-taking and stop-loss thresholds, arguing that this greatly reduces backtest overfitting compared with a brute-force search on the single history. There the generator is one fixed parametric model and the task is calibration; competing generators are not compared.

Koshiyama, Firoozye and Treleaven (2021) come closest to a task-based evaluation of deep generators for trading. They use conditional GANs to calibrate the hyperparameters of trading strategies and to combine strategies into ensembles, testing on 579 assets. They report that the approach can outperform when traditional techniques fail to generate alpha. Their evaluation is task-based, but its question is profitability: does tuning or combining with GAN samples make money? It does not ask whether rankings on synthetic data predict rankings on unseen real data, and it does not compare the GAN against cheap resampling or volatility baselines on that question.

2.6 Gap and Positioning

Taken together, the overfitting literature corrects selection on a single history. The generator literature measures mainly resemblance, and its task-based evaluations concern prediction. The evaluation literature records that no agreed metric exists. Synthetic backtesting has been used to calibrate rules under one parametric model (López de Prado, 2018) and to tune and combine strategies with GANs (Koshiyama, Firoozye and Treleaven, 2021). What is missing is a direct comparison of competing generators, classical and deep, on whether their synthetic backtests rank trading rules the way unseen real data later does.

This project fills that gap at pilot scale: a single asset (USD/EUR), a mainly moving-average rule universe and one test period. Consistent with the efficient-market view that price changes are hard to predict (Fama, 1970), it studies the quality of validation, not the discovery of profitable rules.

3. Methods and Experimental Design

Figure 3.1 summarises the evaluation workflow; the subsections below describe each stage.

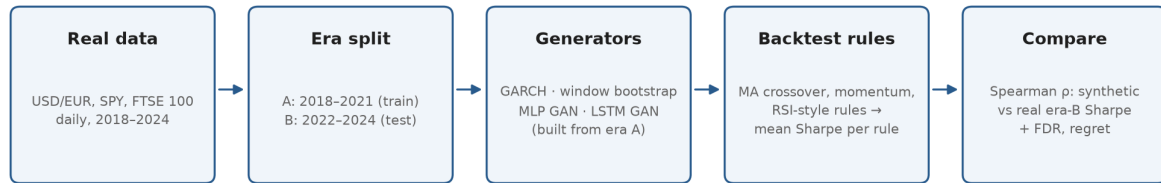


Figure 3.1. Workflow. Generators are built from era-A data; every rule is backtested on synthetic paths and on real era-B data, and the two sets of Sharpe ratios are compared.

3.1 Data

Sources and assets:

- **USD/EUR**: exchange rate ($\text{EUR}=\text{X}$), downloaded from Yahoo Finance (Yahoo Finance, 2026) with the open-source yfinance library (Aroussi, n.d.)
- **SPY**: S&P 500 exchange-traded fund
- **FTSE 100**: UK equity index (^FTSE)
- All daily closing prices, 2018–2024 (7 years), from the same source

Era split (chronological, before windowing):

- **Era-A (2018–2021, training)**: about 1,010–1,040 daily returns per asset (788 USD/EUR windows)
- **Era-B (2022–2024, held-out test)**: about 750–780 daily returns per asset (526 USD/EUR windows)
- Split prevents look-ahead leakage; era-B is entirely unseen during training

Window and return counts for every asset and era are given in Appendix A (Table A.1).

Choice of instruments. The proposal planned hourly cryptocurrency data as the primary series, with a daily cross-asset arm. After the cryptocurrency download failed (Section 5.5), the daily arm became the study. USD/EUR, SPY and the FTSE 100 are liquid daily series without survivorship bias, covering three distinct markets: a major currency pair, a US equity fund and a UK equity index. Because of time constraints, only USD/EUR was evaluated; SPY and the FTSE 100 were prepared for the cross-asset extension (RQ3).

Quote convention. The USD/EUR series is Yahoo Finance's $\text{EUR}=\text{X}$ quote, which gives euros per US dollar (Yahoo labels it USD/EUR). Its log returns are therefore the negative of the conventional EUR/USD rate, and the long-only rules in this study are long the dollar against the euro. Throughout the report, "USD/EUR" refers to this series as downloaded.

Choice of eras. The chronological split places 2018–2021 in era A and 2022–2024 in era B, so that the held-out period is a different market regime rather than a further sample of the same one. Table 3.1 and Figure 3.2 show that this holds for USD/EUR. Era B has higher volatility (7.9% against 6.4% annualised) and stronger volatility clustering (lag-1 autocorrelation of squared returns 0.10 against 0.03), with its most volatile stretch in the first year of era B. For the two equity series the ordering is reversed, because era A contains the 2020 market shock. A four-year training era gives about 1,000 daily returns per asset, enough for 256-day windows, while leaving three years for testing.

Table 3.1. Daily log-return statistics by asset and era. Series rebuilt from the windows (`pipeline/14_data_description.py`, `outputs/data_description.json`).

Asset	Era	Returns	Annualised volatility	Skewness	Excess kurtosis	Max drawdown	Autocorr. of squared returns, lag 1
USD/EUR	A (2018–21)	1,043	6.4%	0.37	2.9	-13.6%	0.03
USD/EUR	B (2022–24)	781	7.9%	0.01	1.2	-14.6%	0.10
SPY	A (2018–21)	1,007	20.9%	-1.05	15.9	-33.7%	0.49
SPY	B (2022–24)	752	17.5%	-0.23	1.8	-24.5%	0.10
FTSE 100	A (2018–21)	1,011	18.3%	-1.22	16.4	-36.6%	0.15
FTSE 100	B (2022–24)	754	12.8%	-0.45	3.5	-11.0%	0.23

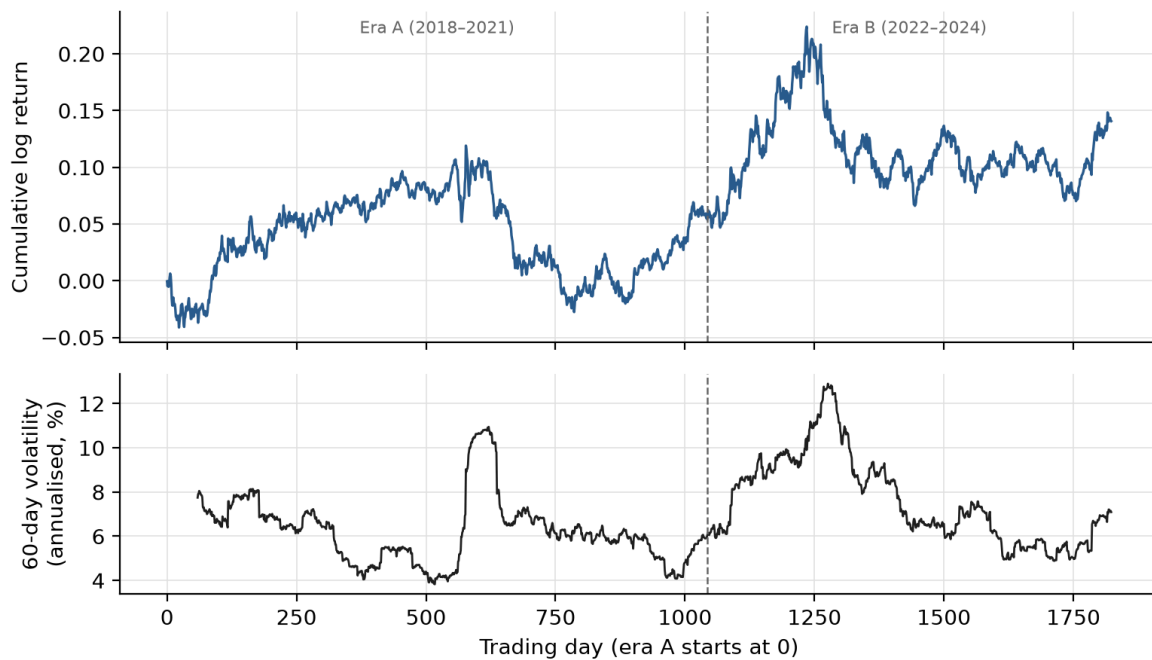


Figure 3.2. USD/EUR (EUR=X) cumulative log return and 60-day annualised volatility; the dashed line marks the era split. The windows carry no dates, so the axis counts trading days.

The table also shows why the pooled fidelity statistics in Table 4.1 are not USD/EUR stylized facts: USD/EUR's excess kurtosis is about 3 in era A and 1 in era B, against 22 for the pooled matrix.

Evaluation focus: Evaluation focused on USD/EUR (columns 0–255 of 768-dim data); SPY and FTSE prepared but not included in RQ2 analysis due to time constraints. This represents a focused case study on a single asset. The three instruments were chosen as liquid daily series without survivorship bias, covering a currency pair, a US equity fund and a UK equity index. The stored data hold one asset per row in a 768-column layout (3 assets × 256 steps); columns belonging to the other two assets are empty and were zero-filled before GAN training and bootstrap resampling (Section 5.4).

Returns and preprocessing:

- Log returns: $r_t = \ln(P_t / P_{t-1})$
- 256-step windows with stride 1 (maximum overlap)
- Underlying return sequences, era-A: 1,043 (USD/EUR), 1,007 (SPY), 1,011 (FTSE 100)
- Underlying return sequences, era-B: 781 (USD/EUR), 752 (SPY), 754 (FTSE 100)

3.2 Windowing Strategy

Why 256 steps?

- Volatility clustering decays slowly, roughly as a power law in the lag (Cont, 2001), so long windows are needed to contain several volatility episodes
- 256 days $\approx$ 1 calendar year; captures multiple volatility cycles

- Aligns with technical analysis conventions (200–260-day moving averages)

Stride: 1 day (maximum overlap)

- Maximises sample count for generative model training
- Standard in deep learning (sliding windows in CNNs, LSTM datasets)

Leakage prevention:

- Windows created *after* era split
- No window contains data from both eras
- Boundary-straddling windows discarded

3.3 Generators Evaluated

Baseline 1: Window-Level Resampling (Nonparametric)

- Resamples entire 256-step windows with replacement
- Generates synthetic data by randomly selecting from empirical windows
- Nonparametric; preserves the empirical statistics of each window
- The RQ1 fidelity run used the same whole-window resampling (`02_baselines.py`)

Baseline 2: GARCH(1,1) (Bollerslev, 1986)

- Parametric volatility model: $h_t = \omega + \alpha \cdot \epsilon^2_{t-1} + \beta \cdot h_{t-1}$
- Parameters: $\omega = 1e-6$, $\alpha = 0.05$, $\beta = 0.94$ (literature defaults)
- Captures volatility clustering; fast, interpretable
- The `fit()` step stores only the sample mean and standard deviation, so parameters were not estimated from the data. The mean and standard deviation are taken over the pooled, zero-filled three-asset matrix, and the same path is copied to all three assets. With these defaults the long-run daily volatility is about 1%, roughly twice that of USD/EUR

Primary: MLP GAN

- Generator: 4-layer MLP (100-dim noise $\rightarrow$ [256, 512, 256] $\rightarrow$ 768-dim tanh output)
- Discriminator: 4-layer MLP with dropout, LeakyReLU
- 50 epochs training; BCEWithLogitsLoss
- Per-epoch losses are stored in the model checkpoint and exported by `13_gan_training_history.py` (Appendix B)

Robustness check: LSTM GAN

- Generator: Bidirectional LSTM layers + linear projection (explicit temporal modelling)
- Discriminator: LSTM + classification head
- 50 epochs training
- Same training data and evaluation as the MLP GAN

3.4 Evaluation Metrics

RQ1: Stylized Facts (Statistical Fidelity)

- Mean autocorrelation of returns over lags 1–20: should be close to 0
- Mean autocorrelation of squared returns over lags 1–20: should be positive (volatility clustering)
- Excess kurtosis: should match the real data
- Wasserstein distance: overall distributional distance

RQ2: Downstream Task (Strategy Ranking Utility)

- For each distinct trading rule (185; 181 for the GAN generators):
 1. Backtest on each real era-B window and average → real Sharpe ratio
 2. Backtest on each synthetic path (1,000 per generator) and average → synthetic Sharpe ratio
- Sharpe ratio here is the per-window mean of trade returns divided by their standard deviation (not annualised)
- Measure: Spearman rank correlation (synthetic vs real Sharpe)
 - $\rho > 0$ means: rules ranked higher on synthetic data tend to rank higher on real data
 - $\rho = 0.0$ means: no correlation; synthetic ranking is useless
 - $\rho = 1.0$ means: perfect rank preservation

3.5 Trading Rules Universe

A parameter sweep produced 500 rule specifications, but only 197 distinct rule names. The evaluator stores results by rule name, so specifications sharing a name (for example, variants differing only in stop-loss level) were merged and received identical Sharpe ratios. Each merged rule is counted once in this report. After also removing rules with no trades (undefined Sharpe), 185 distinct rules remain: 180 MA crossover, 4 momentum and 1 RSI. The GAN evaluations produced no valid momentum results, leaving 181.

Entry signals:

- MA crossover (fast MA $\in [10, 30]$, slow MA $\in [50, 150]$), with moving averages computed on the return series: 480 specifications
- Momentum (threshold $\in [0.01, 0.05]$): 16 specifications
- RSI-style rule (entry when the day's return falls below a percentile of returns, threshold $\in [25, 40]$): 4 specifications. In the as-run code the percentile was taken over the whole window, including later days, which is a look-ahead; the corrected code uses past returns only

Exit signals:

- Time-based hold (bars $\in [5, 20]$): simple exit after N bars
- Profit-taking (target $\in [0.01, 0.10]$): exit when profit $\geq$ target
- Stop-loss (threshold $\in [-0.05, -0.01]$): exit when loss $\leq$ threshold

- Combined: exit on first of time/profit/loss

All rules are long-only. The as-run backtester allowed a new trade to open while another was still held; the corrected backtester holds one position at a time. All parameters were checked to be finite and within the stated bounds. Appendix C summarises the resulting rule universe.

3.6 Testing and Verification

The pipeline has a pytest suite covering data integrity, leakage prevention, the baseline generators, rule generation, the GANs and an end-to-end pipeline check.

During the final review the suite was first repaired. The data-pipeline, strategy and GAN test modules had been silently skipped because they imported modules under names that did not match the numbered pipeline files; a test configuration now maps those names. Five tests contained errors of their own (an outdated constructor argument, a numeric check applied to text parameters, a diversity check on too small a sample, an era-boundary check that misread the split function's argument, and a bootstrap check that compared rows from different assets); these were corrected.

Once every test ran, the suite exposed genuine defects in the code that produced the results reported here:

- trade returns were the difference of two single-day returns rather than the holding-period return, and moving averages were computed on returns rather than prices;
- rule names were not unique, so results stored by name were merged (Section 3.5);
- GARCH produced missing values when fitted on input containing gaps, and was fitted on the pooled, zero-filled matrix;
- the RSI-style rule chose its threshold from the whole window, so it used future returns;
- GAN samples were drawn without a fixed PyTorch seed, so GAN results varied between runs.

These defects have since been fixed in the pipeline code, but the results in this report have not been regenerated with the fixed code. The reported results come from the scripts as they ran, archived in `archive/pipeline_as_run_2026-10-05/`. Section 5.4 discusses how the defects may affect them. The corrected backtester also changes the Sharpe definition to the mean over standard deviation of the strategy's daily returns (position $\times$ return), because with price-based moving averages many rules trade only once per window, and a per-trade Sharpe ratio is then undefined. With the fixes, the suite gives 78 passed and 2 expected failures (run on 2026-10-05). The two expected failures concern the GANs (output scale, and non-deterministic training under a fixed seed), which can only be addressed by retraining. Both GAN scripts now accept an `--eur-only` option that trains on USD/EUR windows without zero-filling and saves a separate model with its loss history; the evaluation uses those models when they exist.

3.7 Reproducibility

- **Environment.** Python 3.11.7. `requirements.txt` pins the direct dependencies (numpy 2.4.6, pandas 3.0.6, scipy 1.17.1, torch 2.14.0, yfinance 1.7.0 and others); `requirements-lock.txt` is a full `pip freeze` of the environment; `requirements-build.txt` adds the notebook and PDF tools. No credentials or environment variables are needed.
- **Data.** The frozen dataset is the pair of windowed-return files `windowed_data_era_a.parquet` and `windowed_data_era_b.parquet`, tracked in version control with their SHA-256 hashes recorded in the project README. Raw prices were not saved, the download date and yfinance version at download time were not recorded, and Yahoo data can be revised, so re-downloading may not reproduce these files exactly. Prices are adjusted closes (yfinance's default `auto_adjust=True`).
- **Randomness.** Seeds are fixed (`numpy.random.seed(42)` , `torch.manual_seed(42)`), which makes resampling and GARCH simulation repeatable. In the as-run evaluation the GAN samples were drawn without a PyTorch seed; the corrected scripts seed them. GAN training is not deterministic (Section 3.6), so retraining gives different models; the trained weights used here are saved (`gan_model.pt` , `gan_model_lstm_fixed.pt`).
- **Running.** The README lists the scripts in order; Appendix E gives the code that reproduces the main table and the build commands. The RQ2 statistics and figures in this report come from `10_distinct_rule_rq2.py` and `11_report_figures.py`, which read saved outputs and run in under a minute. `GENERATE_PDF.sh` executes this notebook and exports the PDF.

3.8 Implementation

The study is implemented as a Python 3.11 pipeline of numbered scripts (Table 3.2). Each script reads from and writes to `outputs/`, so stages can be rerun independently. Numerical work uses NumPy, pandas and SciPy; the GANs use PyTorch; data are downloaded with yfinance and stored as Parquet files. All computation ran on a laptop CPU: the MLP GAN trains in about two minutes, the LSTM GAN in about two hours, and backtesting every rule for one generator takes about a minute.

Table 3.2. Pipeline stages, scripts and main outputs.

Stage	Script	Main output
Data collection and windowing	<code>01_data_pipeline.py</code>	<code>windowed_data_era_{a,b}.parquet</code>
Classical generators and RQ1 fidelity	<code>02_baselines.py</code>	<code>fidelity_results.csv</code>
Rule universe	<code>03_strategies.py</code>	rule list (in memory)
GAN training	<code>04_gan_generator.py</code> , <code>04b_gan_generator_lstm_FIXED.py</code>	<code>gan_model.pt</code> , <code>gan_model_lstm_fixed.pt</code>
Backtesting (RQ2)	<code>05_evaluation.py</code> , <code>05b_evaluation_lstm_comparison.py</code>	<code>evaluation_results*.csv</code>
Paired comparison of generators	<code>12_paired_rank_tests.py</code>	<code>paired_rank_tests.json</code>
GAN loss histories	<code>13_gan_training_history.py</code>	<code>gan_training_history.json</code>
RQ2 statistics and figures	<code>10_distinct_rule_rq2.py</code> , <code>11_report_figures.py</code>	<code>rq2_distinct_rules.json</code> , figures
Data description	<code>14_data_description.py</code>	<code>data_description.json</code> , Figure 3.2
Report build	<code>GENERATE_PDF.sh</code> , <code>tools/</code>	PDF and HTML

Data layout. Each stored window holds 256 daily log returns for one asset in a 768-column layout (three assets × 256 steps), with an asset label and a window identifier; the other assets' columns are empty. Helper functions in `05_evaluation.py` select single-asset windows (`asset_windows`) and rebuild the underlying series from overlapping windows (`reconstruct_series`).

Backtester. `StrategyEvaluator` applies one rule to one window. It computes the entry signal (a moving-average crossover, a momentum threshold or a percentile rule), applies the exit (a fixed holding period, or a profit target and stop loss within a maximum holding period) and returns a Sharpe ratio. Values are then averaged over all real or synthetic windows. Appendix E.2 compares the version that produced the reported results with the corrected version.

Rule generation. `generate_strategies` sweeps the parameter grids in a fixed order and stops at 500 specifications. Because moving-average rules come first, the Bollinger-band rules in the grid are never reached (Section 5.5).

Quality controls. The test suite (Section 3.6), fixed seeds, pinned dependencies (Section 3.7) and an archived copy of the scripts as run (Appendix D) support verification. Every module, class and function in the pipeline carries a docstring describing its inputs, outputs and conventions.

4. Results

4.1 RQ1: Stylized Facts Fidelity

Fidelity was measured on the era-A training matrix (2,296 rows × 768 columns), which pools the three assets. The "real" statistics below therefore describe the pooled training input, not USD/EUR alone, and are useful only for comparing a generator with the data it was built from.

Table 4.1. RQ1 fidelity statistics: pooled era-A training data against the window bootstrap.

Metric	Real (pooled)	Window bootstrap	Deviation
Mean autocorr. of returns, lags 1–20	-0.00177	-0.00178	0.2%
Kurtosis	22.43	21.99	2.0%
Skewness	-1.232	-1.223	0.8%
Mean autocorr. of squared returns, lags 1–20	0.2192	0.2190	0.1%

Source: `outputs/fidelity_metrics_summary.json`.

Interpretation: As Table 4.1 shows, the window bootstrap resamples whole observed windows, so it reproduces these moments almost exactly by construction. The statistics were computed by concatenating overlapping windows from all three assets, so they are not clean single-asset stylized facts.

GARCH and GAN fidelity. The fidelity script returned non-finite values for GARCH and was not run for either GAN, so no RQ1 comparison is available for them. Earlier drafts reported GARCH and GAN values that do not appear in any output file; they have been removed. RQ1 is therefore answered only for the bootstrap baseline.

Generated paths. Figure 4.1 shows five cumulative-return paths from each source. The window bootstrap replays real era-A windows, so its paths look like real data. GARCH paths have the right shape but are more volatile than USD/EUR, because its parameters were not calibrated. The MLP GAN's paths are almost flat: across 500 draws its median daily standard deviation is 0.0005, against 0.0047 for real era-B USD/EUR windows. The LSTM GAN's typical spread (0.0027) is closer to the real one, but some of its paths are near-straight drift lines, a sign of mode collapse. These are qualitative observations rather than a substitute for the missing fidelity statistics.

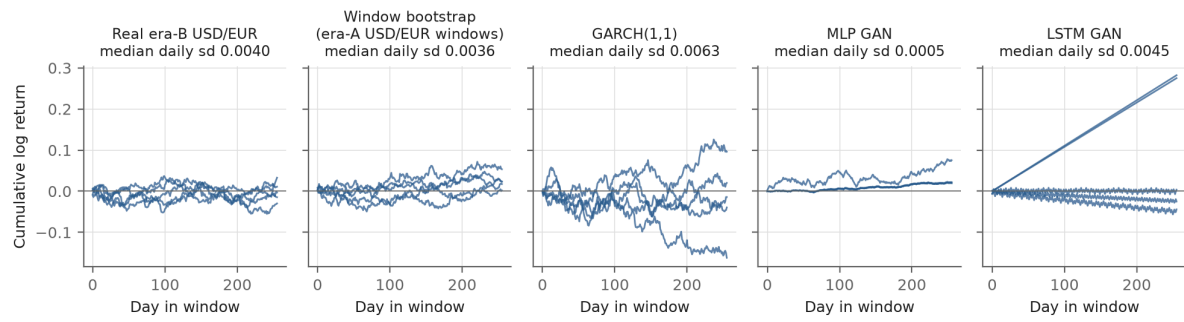


Figure 4.1. Five cumulative-return paths from each source (USD/EUR columns only). Panel titles give the median daily standard deviation of the paths shown.

4.2 RQ2: Downstream Task-Based Utility (Primary Result)

Each rule's mean Sharpe ratio on synthetic paths was compared with its mean Sharpe ratio on the 526 real era-B USD/EUR windows. Statistics are computed on distinct rules (Section 3.5) by `pipeline/10_distinct_rule_rq2.py`, which writes

`outputs/rq2_distinct_rules.json`; the code in Appendix E.1 reproduces Table 4.2.

Table 4.2. Spearman rank correlation between synthetic and real era-B Sharpe ratios, distinct rules.

Generator	Spearman ρ	95% CI (Fisher z)	p-value	n
GARCH(1,1)	0.536	[0.424, 0.631]	<0.001	185
Window bootstrap	0.337	[0.203, 0.459]	<0.001	185
LSTM GAN	0.299	[0.161, 0.427]	<0.001	181
MLP GAN	0.189	[0.045, 0.326]	0.011	181

Reading Table 4.2. Every generator's ρ is significantly above zero, so synthetic backtests carry some information about real out-of-sample ranking. The p-values test each ρ against zero, not one generator against another. Overlap between these marginal intervals is only a conservative guide to pairwise differences, because all four correlations are computed on the same rules. Section 4.4 tests the differences directly. On the marginal intervals alone, only the GARCH–MLP GAN gap is resolved.

Effect of merged duplicate rules. Computed on all 484–488 rows, the same correlations are higher and their intervals narrower (GARCH 0.567, bootstrap 0.384, LSTM GAN 0.382, MLP GAN 0.172), because merged rules are counted up to four times. The distinct-rule figures are reported throughout.

Figure 4.2 shows the same estimates with their confidence intervals, and Figure 4.3 shows the rank agreement behind each correlation.

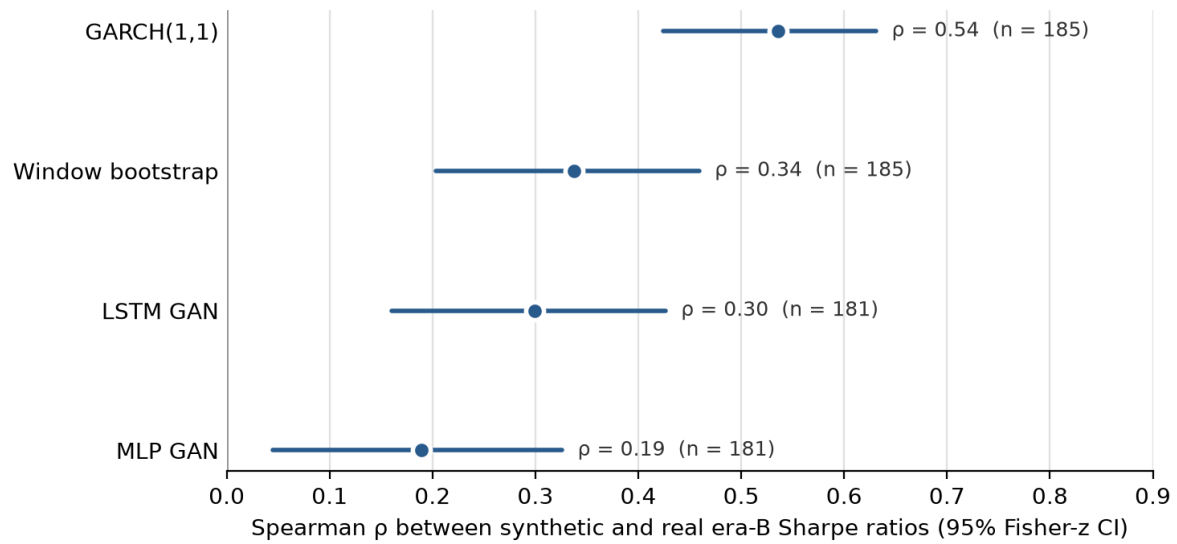


Figure 4.2. Spearman ρ between synthetic and real era-B Sharpe ratios, with 95% Fisher-z confidence intervals, distinct rules.

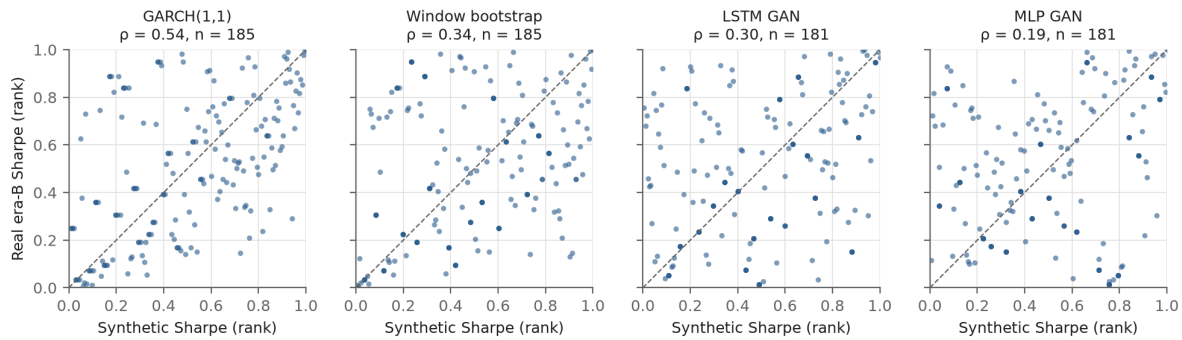


Figure 4.3. Percentile rank of each rule's real era-B Sharpe ratio against its rank on synthetic data. Points on the dashed line would mean perfect agreement; ρ summarises the scatter.

Walk-forward baseline. RQ2 asks for a comparison with single-history walk-forward validation. The pipeline's walk-forward script (`07_walkforward_baseline.py`) does not backtest on era-A: it creates "era-A predictions" by adding random noise of a chosen size to the real era-B Sharpe ratios, so its correlation (0.446) is set by that choice and carries no evidence. It is excluded from the results. A genuine walk-forward baseline has since been implemented (Section 3.6) but not yet run, so this part of RQ2 remains unanswered.

4.3 RQ2 Secondary Metrics: False Discovery, Optimism and Selection Regret

Three further metrics describe the decisions a practitioner would make from synthetic backtests. A rule is *endorsed* if its synthetic Sharpe ratio exceeds 0.1, and counted as *working* if its real era-B Sharpe ratio also exceeds 0.1. Optimism is the synthetic minus the real Sharpe ratio. Selection regret is the best real Sharpe ratio (2.29) minus the real Sharpe ratio of the rule ranked first on synthetic data.

Table 4.3. Decision metrics by generator, distinct rules (endorsement threshold: Sharpe ratio > 0.1).

Generator	Endorsed	Endorsed that work	FDR	Mean optimism	Median optimism	Top pick's real Sharpe	Selection regret
GARCH(1,1)	1	1	0%	-0.13	-0.04	2.29	0.00
Window bootstrap	1	1	0%	-0.06	0.10	2.29	0.00
LSTM GAN	27	1	96%	unstable	-6.07	-0.08	2.37
MLP GAN	1	1	0%	-0.17	-0.09	2.29	0.00

Source: `outputs/rq2_distinct_rules.json`.

Interpretation. In Table 4.3, GARCH, the bootstrap and the MLP GAN each endorse a single rule, which is also their top pick and the best rule on real data. Their 0% false-discovery rate and zero regret therefore rest on one rule and do not discriminate between them. The LSTM GAN behaves differently: it endorses 27 rules, of which one works out-of-sample, and its top-ranked rule loses money on real data. Its mean optimism is dominated by a few extreme synthetic Sharpe ratios, a sign of unstable generated paths (Appendix B), so the median is reported. GARCH and the MLP GAN show small negative median optimism (their synthetic backtests slightly understate real performance), and the bootstrap a small positive one; these differences are small relative to the spread.

Overall. The secondary metrics separate the LSTM GAN from the other generators but do not separate GARCH, the bootstrap and the MLP GAN. They neither contradict nor independently confirm the rank-correlation ordering.

4.4 Robustness and Paired Comparison of Generators

To compare generators directly, the 181 distinct rules for which all four generators have a defined Sharpe ratio were resampled with replacement 1,000 times, and every generator's ρ was recomputed on each resample (`pipeline/12_paired_rank_tests.py` , `outputs/paired_rank_tests.json`). Because each resample is shared by all generators, differences between them can be tested directly. This replaces an earlier three-generator check (`06_bootstrap_resampling_validation.py`), whose intervals it reproduces closely.

Table 4.4. Spearman ρ on the 181 common rules, with 95% percentile intervals from 1,000 resamples.

Generator	ρ (n = 181)	95% interval
GARCH(1,1)	0.508	[0.387, 0.629]
Window bootstrap	0.293	[0.153, 0.441]
LSTM GAN	0.299	[0.147, 0.446]
MLP GAN	0.189	[0.030, 0.347]

Table 4.5. Paired differences in ρ across the same resamples: mean, 95% percentile interval and share of resamples in which the difference is zero or negative.

Difference	Mean	95% interval	Resamples ≤ 0
GARCH(1,1) – Window bootstrap	+0.213	[0.120, 0.309]	0.0%
GARCH(1,1) – LSTM GAN	+0.208	[0.052, 0.378]	0.5%
GARCH(1,1) – MLP GAN	+0.322	[0.128, 0.517]	0.0%
Window bootstrap – LSTM GAN	-0.005	[-0.199, 0.181]	53.3%
Window bootstrap – MLP GAN	+0.109	[-0.107, 0.311]	15.4%
LSTM GAN – MLP GAN	+0.114	[-0.053, 0.271]	9.7%

Source: `outputs/paired_rank_tests.json` .

Interpretation. Table 4.5 shows GARCH ahead of the bootstrap, the LSTM GAN and the MLP GAN, with every 95% interval above zero, whereas no pair among the other three generators is separated. The paired test is more informative than comparing the separate intervals in Table 4.4, because a resample that favours one generator tends to favour the others too. The test treats rules as independent; many rules are close variants of one another (Section 5.4), so the effective number of rules is smaller and these intervals are probably too narrow.

4.5 Sensitivity by Rule Family

Table 4.6. Spearman ρ by rule family, distinct rules.

Rule family	Distinct rules	GARCH ρ	Bootstrap ρ	LSTM GAN ρ	MLP GAN ρ
MA crossover	180	0.500	0.281	0.288	0.176
Momentum	4 (GARCH and bootstrap only)	—	—	—	—
RSI-style	1	—	—	—	—

Source: `outputs/rq2_distinct_rules.json`. Momentum and RSI-style rules are too few for a correlation.

Interpretation. As Table 4.6 shows, the results are effectively results for MA crossover rules, which make up 180 of the 185 distinct rules. Within that family GARCH remains highest and the MLP GAN lowest, while the bootstrap (0.281) and the LSTM GAN (0.288) swap places but are almost equal; GARCH's ρ falls slightly. Nothing can be concluded about other signal families. Earlier drafts reported an RSI correlation of 1.0 on four rules; those four specifications are a single merged rule, so that figure was not meaningful.

4.6 Summary of Results

Table 4.7 summarises the findings for each research question.

Table 4.7. Summary of findings by research question.

RQ	Question	Finding
RQ1	Stylized-facts fidelity	The window bootstrap reproduces training-data moments almost exactly. GARCH and GAN fidelity were not measured.
RQ2 (primary)	Downstream utility	All generators carry positive ranking signal (ρ 0.19–0.54). GARCH is highest and, in paired comparisons, ahead of each other generator; the other three cannot be separated. Evidence is limited to MA crossover rules.
RQ3	Cross-regime and cross-asset	Deferred.

5. Discussion

5.1 Why Might GARCH Rank Strategies Better than the Bootstrap?

The window bootstrap matches the training moments closely (RQ1), yet ranks rules less well than GARCH (p 0.34 vs 0.54), a gap the paired comparison in Section 4.4 supports. One explanation is that the RQ2 bootstrap resamples whole 256-day windows from era-A, so every synthetic path is an era-A history. Rules that suited 2018–2021 are rewarded whatever their later performance, which makes the bootstrap behave like an in-sample backtest.

GARCH instead generates new paths whose volatility clusters through the recursion $h_t = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta \cdot h_{t-1}$. With the default parameters ($\alpha + \beta = 0.99$), shocks decay slowly, so each path contains long volatility episodes and trend rules are tested under varied conditions rather than on a replay of one history.

This explanation is a hypothesis. The study does not isolate which property of the GARCH paths drives the ranking, and the GARCH scale was not calibrated to USD/EUR (Section 5.4).

Key insight: close moment matching did not produce the best ranking. Task-based evaluation can reveal differences that fidelity metrics do not.

5.2 Why the Deep Generators Underperform

Both GANs rank rules less well than GARCH (MLP 0.19, LSTM 0.30). The LSTM's p is higher than the MLP's, but the paired comparison cannot separate the two (Section 4.4); its synthetic paths produce extreme Sharpe ratios, and its training losses stayed at their chance-level values throughout (Appendix B). Plausible reasons for the weaker performance:

1. **Limited data:** 788 heavily overlapping USD/EUR training windows contain only about four non-overlapping ones.
2. **Objective mismatch:** the adversarial loss does not target the temporal features that trading rules exploit.
3. **Zero-filled inputs:** both GANs were trained on the pooled 768-column matrix in which two thirds of each row is zero-filled, so they learn artificial flat segments as well as real returns.
4. **No tuning:** no hyperparameter search was carried out.
5. **Wrong output scale:** the MLP GAN's paths have roughly a tenth of the real daily volatility, and some LSTM paths collapse to near-straight lines (Figure 4.1). On such paths, trade returns have little variation, so Sharpe ratios become erratic. This may also explain the LSTM GAN's extreme optimism values in Section 4.3.
6. **Training dynamics:** the logged losses show the MLP discriminator overpowering the generator, and the LSTM making no measurable progress (Appendix B).

GARCH, by contrast, has three parameters and builds volatility clustering in by construction. The comparison shows that these GANs, trained this way, underperform; it does not show

that deep generators cannot work for this task.

5.3 Relation to Prior Work

Fidelity and usefulness. Assefa et al. (2019) caution that resemblance measures cannot be relied on as evidence of realism. The results add a related, task-based observation: the window bootstrap, which matches the training moments almost exactly, did not give the best rule ranking. This supports evaluating generators on the decision they are meant to inform, not only on resemblance. The evidence is limited, though: GARCH's lead over the bootstrap holds in a paired comparison but rests on many closely related rules, and only one asset and one rule family were tested.

Classical against deep generators. Quant GANs (Wiese et al., 2020) and GAN models of financial returns (Takahashi, Chen and Tanaka-Ishii, 2019) show that deep generators can reproduce stylized facts. Here, a GARCH(1,1) with default parameters ranked rules at least as well as either GAN. This is not evidence against those architectures, which were not implemented here: the MLP and LSTM GANs in this study were trained on zero-filled multi-asset inputs and produced paths of the wrong scale (Figure 4.1). The defensible conclusion is that a deep generator should be required to beat cheap classical baselines on the downstream task before its extra cost is justified.

Task-based GAN evaluation. Koshiyama, Firoozye and Treleaven (2021) found that conditional GANs can help calibrate and combine strategies. Their setting differs from this one in the generator (conditional), the task (calibration and ensembling) and the scale (579 assets), so the weaker GAN results here do not contradict theirs; they measure a different use of synthetic data.

Overfitting corrections. A rank correlation of about 0.5 means synthetic backtests carry real but partial information about out-of-sample ranking. Synthetic validation is therefore best seen as a complement to the single-history corrections of White (2000), Bailey and López de Prado (2014) and Bailey et al. (2017), not a substitute for them.

A useful negative result. The proposal anticipated that a negative result would be as informative as a positive one. For the deep generators, that is what this study found: neither GAN beat the cheap classical baselines on the downstream task, and both were beaten by a three-parameter GARCH model. Within the limits of this pilot, this is evidence that generative complexity does not automatically translate into better validation decisions, and that cheap baselines should be the default comparison for any new market generator. A fairer test would use well-trained deep generators (Section 6.4).

The single-history problem. The evaluation itself inherits the limitation Assefa et al. (2019) describe for backtest-style evaluation: rankings are judged against one realised test period, so a different period could give a different ordering.

5.4 Limitations

1. **Backtester trade returns (fixed in code, not rerun).** Trade returns are computed as the difference between the single-period returns on the exit and entry days (`trade_return = returns[exit_idx] - returns[entry_idx]` in the as-run `05_evaluation.py`; Appendix E.2), not as the cumulative return over the holding period. Profit-target and stop-loss checks use the same quantity, and moving averages are computed on returns rather than prices. All Sharpe ratios, real and synthetic, share this definition, so the comparison between generators is internally consistent. However, the Sharpe values are not economic trading returns, and the ranking could change once the backtester is corrected.
2. **Merged rule specifications (fixed in code, not rerun).** Results are stored by rule name, and only 197 of the 500 specifications have distinct names. The analysis therefore covers 185 distinct rules, not 500.
3. **GARCH not fitted (variance targeting added in code, not rerun).** GARCH uses literature-default parameters, and its scale was not calibrated to USD/EUR. `garch_parameters.json` lists fitted-looking values, but no script produces them, and its stated half-life (35 days) does not match its own parameters (about 57 days), so they are not used.
4. **Incomplete RQ1.** Fidelity statistics exist only for the window bootstrap, and they describe the pooled three-asset matrix rather than USD/EUR. The corrected fidelity script computes them on USD/EUR for every generator but has not been run.
5. **No walk-forward baseline (implemented in code, not run).** The walk-forward script simulates its predictions from the era-B outcomes and is excluded (Section 4.2). RQ2's comparison with walk-forward validation is therefore not answered.
6. **Data construction.** The three assets are stored in a 768-column layout in which the other assets' columns are zero-filled for GAN training and bootstrap resampling. The stored windows carry no dates, and the price-adjustment setting of the download was left implicit.
7. **Scope.** The study covers MA crossover rules (180 of 185), a single asset (USD/EUR) and a single test period (2022–2024).
8. **Overlapping windows.** Stride-1 windows share 255 of 256 observations, so the effective sample size is far smaller than the window count.
9. **GAN training.** No hyperparameter search was done, the GAN training-summary file cannot be traced to any computation (Appendix B), and LSTM generation was numerically unstable.
10. **Correlated rules.** Many rules differ only in one exit parameter, so their Sharpe ratios are strongly correlated. The resampling intervals in Section 4.4 treat rules as independent and are therefore probably too narrow.
11. **Further as-run defects.** The RSI-style rule used future returns within each window (one distinct rule), and GAN samples were not seeded, so the GAN correlations would vary slightly between runs. Both are fixed in code but not rerun.

Assessment. Limitations 1, 3 and 5 affect the internal validity of the RQ2 comparison; 6–8 limit how far the findings generalise. The results are best treated as a pilot that demonstrates the task-based evaluation framework and motivates a corrected rerun.

5.5 Changes from the Proposal

Table 5.1 sets out where the delivered study departs from the proposal.

Table 5.1. Planned and delivered design.

Element	Proposal	This study
Data	Hourly BTC/USDT and ETH/USDT as primary data; daily EUR/USD, SPY and a FTSE 100 fund for a cross-asset arm	Daily USD/EUR only (Yahoo $\text{EUR}=\text{X}$, quoted as euros per dollar); SPY and the FTSE 100 index, not the tradable fund, prepared but not evaluated
Deep generator	Temporal-convolutional GAN in the Quant GAN style	MLP GAN and LSTM GAN
Resampling baseline	Stationary (block) bootstrap	Whole-window bootstrap
Rule families	Moving-average crossover, RSI, Bollinger bands	Moving-average crossover, momentum, RSI-style; Bollinger rules not reached
False-discovery threshold	Sharpe ratio significantly above zero at the 5% level	Fixed Sharpe threshold of 0.1
Synthetic ground-truth control	Known data-generating processes with a known best rule	Not implemented
Transaction costs	Exchange fees and spreads	Not modelled
Comparing methods	Tests for differences in predictive accuracy	Paired resampling of differences in p (Section 4.4)
Walk-forward baseline	Single-history walk-forward validation	Simulated placeholder; replaced in code but not yet run
RQ3	Extension	Deferred

The data change followed a failed attempt to collect the hourly cryptocurrency data through the ccxt library; daily data for the cross-asset instruments were already available. The proposal also expected GARCH to be the weaker comparator, because with near-independent innovations it cannot reproduce conditional return predictability. The result was the opposite: GARCH gave the highest rank agreement. Section 5.1 offers one explanation, and the outcome is a reminder not to prejudice the comparison.

Response to proposal feedback. The feedback asked for:

- a conceptual workflow diagram (now Figure 3.1);
- critical synthesis across studies, including recent diffusion-based work (Section 2);
- justification of the window length (Section 3.2) and of the assets (Section 3.1).

The feedback allowed the cross-asset analysis to remain optional; it is deferred with RQ3.

5.6 Reflection

What worked. The task-based framing turned a vague question (is synthetic data useful?) into a measurable one, and the classical baselines gave the comparison a meaningful reference point. The chronological split and the saved, hashed datasets made the analysis auditable, which is what allowed its defects to be traced.

What went wrong. Several defects in the pipeline that produced the reported results were found only during the final review:

- trade returns were computed incorrectly;
- rule specifications that shared a name were merged;
- the walk-forward baseline was a simulation;
- a GARCH parameter file did not come from any script;
- the GANs were trained on zero-filled multi-asset rows.

Three test modules that would have exposed some of these had been silently skipped because of import errors, so a passing test run gave false assurance. Most of the defects share one cause: outputs were checked for plausibility (finite values, sensible ranges) rather than against independent calculations or invariants.

What would be done differently.

- Run the full test suite automatically from the start, treating skipped tests as failures.
- Assert invariants such as unique identifiers where the data are created.
- Test numerical routines against hand-computed examples, as `tests/test_known_defects.py` now does.
- Keep placeholder results out of the main pipeline.
- Record the code version that produced every output file.
- Plot generated paths early; Figure 4.1 would have revealed the GANs' scale problem before any downstream analysis.

Scope. The project attempted more than the time allowed: three assets, four generators and three research questions. A narrower design, with one asset, fewer generators and a corrected backtester checked twice, would have produced firmer conclusions within the same time.

6. Conclusions

6.1 Answering the Research Questions

RQ1 (stylized-facts fidelity). The window bootstrap reproduces the moments of its training data almost exactly. Fidelity for GARCH and the GANs was not measured, so RQ1 is only partly answered.

RQ2 (downstream utility, primary). Synthetic backtests from all four generators carry positive information about real out-of-sample rule rankings (ρ between 0.19 and 0.54 on distinct rules). GARCH gives the highest rank agreement ($\rho = 0.54$, 95% CI 0.42–0.63). In paired comparisons on the same rules it is ahead of each of the other three generators, which cannot be separated from one another; these intervals are probably too narrow because many rules are close variants. The secondary metrics separate only the LSTM GAN, which endorses many rules that fail out-of-sample. The comparison with walk-forward validation was not carried out. All conclusions are limited to MA crossover rules on USD/EUR in 2022–2024 and are conditional on the backtester defect described in Section 5.4.

RQ3 (cross-regime and cross-asset). Deferred.

6.2 Contribution

Methodological: a task-based framework that scores generators on how well synthetic backtests rank trading rules, rather than on fidelity alone.

Empirical: a pilot comparison in which a simple GARCH(1,1) model gave the highest rank agreement and a closely moment-matched bootstrap did not, consistent with the view that fidelity and decision usefulness can diverge.

Negative result: the deep generators, as implemented here, did not beat cheap classical baselines. That is useful to practitioners weighing tooling costs, although it should be retested with better-trained models.

6.3 Practical Implications

- For MA crossover rules, a GARCH(1,1) generator is a cheap, reasonable starting point for synthetic strategy screening, though the evidence here is preliminary.
- Synthetic validation should complement, not replace, out-of-sample testing on real data.
- Generators for this purpose should be evaluated on the downstream task as well as on fidelity metrics.

6.4 Future Work

1. Rerun RQ1 and RQ2 with the corrected pipeline (holding-period returns, price-based moving averages, unique rule names, single-asset baselines), and run the walk-forward

baseline already implemented (Section 3.6).

2. Widen the rule universe beyond moving-average crossover rules, including the Bollinger-band rules not reached in this study.
3. Fit GARCH by maximum likelihood on USD/EUR rather than by variance targeting with default α and β .
4. Retrain the GANs on single-asset data without zero-filling, with logged training losses, hyperparameter search and repeated runs, and add a diffusion-based generator (Section 2.4).
5. Add the synthetic ground-truth control and the transaction costs planned in the proposal (Section 5.5).
6. Address RQ3: stratify era-B by realised volatility, and extend to SPY and the FTSE 100.

7. Ethical Considerations

Data and participants. The study uses only publicly available market prices for USD/EUR, SPY and the FTSE 100, downloaded from Yahoo Finance with the open-source yfinance library (Yahoo Finance, 2026; Aroussi, n.d.). No human participants were involved, and no personal or confidential data were collected, so the work falls outside human-participant research ethics.

Terms of use and redistribution. yfinance is an independent tool, neither affiliated with nor endorsed by Yahoo. The data were used for non-commercial academic research. Raw price files were not retained; only derived windowed returns are stored in the project repository.

Credentials and security. Access to an IG demo account was tested at the proposal stage, with credentials kept in an untracked environment file excluded from version control. No IG data were used for the reported results, and no credentials appear in the code, outputs or report.

Responsible use and reporting. The project makes no investment recommendation and trades no capital. Synthetic data carries a mild dual-use risk: it could be used to overstate a strategy's robustness. The project addresses that risk by measuring when synthetic validation does and does not predict real performance, and by reporting its own limitations openly, including the pipeline defects found during the final review (Sections 3.6 and 5.4).

Reproducibility and research integrity. Code, tests, pinned dependencies and a hashed copy of the dataset are provided (Sections 3.6–3.7). The scripts are also archived exactly as they produced the reported results (Appendix D), so the findings can be checked independently.

8. References

- Aroussi, R. (n.d.) *yfinance: download market data from Yahoo! Finance's API* [Python library], version 1.7.0. Available at: <https://github.com/ranaroussi/yfinance> (Accessed: 5 October 2026).
- Assefa, S., Dervovic, D., Mahfouz, M., Balch, T., Reddy, P. and Veloso, M. (2019) 'Generating synthetic data in finance: opportunities, challenges and pitfalls', *NeurIPS 2019 Workshop on AI in Financial Services*. Available at SSRN: <https://ssrn.com/abstract=3634235>. (A revised version appeared in the *Proceedings of the First ACM International Conference on AI in Finance*, 2020, doi:10.1145/3383455.3422554, with R.E. Tillman in place of T. Balch.)
- Bailey, D.H., Borwein, J., López de Prado, M. and Zhu, Q.J. (2017) 'The probability of backtest overfitting', *Journal of Computational Finance*, 20(4), pp. 39–69.
- Bailey, D.H. and López de Prado, M. (2014) 'The deflated Sharpe ratio: correcting for selection bias, backtest overfitting, and non-normality', *Journal of Portfolio Management*, 40(5), pp. 94–107.
- Bollerslev, T. (1986) 'Generalized autoregressive conditional heteroskedasticity', *Journal of Econometrics*, 31(3), pp. 307–327.
- Cont, R. (2001) 'Empirical properties of asset returns: stylized facts and statistical issues', *Quantitative Finance*, 1(2), pp. 223–236.
- Fama, E.F. (1970) 'Efficient capital markets: a review of theory and empirical work', *The Journal of Finance*, 25(2), pp. 383–417.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) 'Generative adversarial nets', *Advances in Neural Information Processing Systems*, 27, pp. 2672–2680.
- Harvey, C.R., Liu, Y. and Zhu, H. (2016) '... and the cross-section of expected returns', *The Review of Financial Studies*, 29(1), pp. 5–68.
- Ho, J., Jain, A. and Abbeel, P. (2020) 'Denoising diffusion probabilistic models', *Advances in Neural Information Processing Systems*, 33, pp. 6840–6851.
- Huang, H., Chen, M. and Qiao, X. (2024) 'Generative learning for financial time series with irregular and scale-invariant patterns', in *International Conference on Learning Representations (ICLR 2024)*.
- Koshiyama, A., Firoozye, N. and Treleaven, P. (2021) 'Generative adversarial networks for financial trading strategies fine-tuning and combination', *Quantitative Finance*, 21(5), pp. 797–813.
- López de Prado, M. (2018) *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley.

Politis, D.N. and Romano, J.P. (1994) 'The stationary bootstrap', *Journal of the American Statistical Association*, 89(428), pp. 1303–1313.

Takahashi, S., Chen, Y. and Tanaka-Ishii, K. (2019) 'Modeling financial time-series with generative adversarial networks', *Physica A: Statistical Mechanics and its Applications*, 527, 121261.

White, H. (2000) 'A reality check for data snooping', *Econometrica*, 68(5), pp. 1097–1126.

Wiese, M., Knobloch, R., Korn, R. and Kretschmer, P. (2020) 'Quant GANs: deep generation of financial time series', *Quantitative Finance*, 20(9), pp. 1419–1440.

Yahoo Finance (2026) *Historical daily prices for EUR=X, SPY and ^FTSE, 2018–2024*. Available at: <https://finance.yahoo.com> (Accessed: September 2026).

Yoon, J., Jarrett, D. and van der Schaar, M. (2019) 'Time-series generative adversarial networks', *Advances in Neural Information Processing Systems*, 32.

Yuan, X. and Qiao, Y. (2024) 'Diffusion-TS: interpretable diffusion for general time series generation', in *International Conference on Learning Representations (ICLR 2024)*.

9. Appendices

A. Data Summary

Table A.1. Windows (256 daily log returns, stride 1) and underlying return counts by asset and era.

	Asset	Era-A windows	Era-B windows	Era-A returns	Era-B returns
	USD/EUR	788	526	1,043	781
	SPY	752	497	1,007	752
	FTSE 100	756	499	1,011	754

Windows are 256 daily log returns with stride 1 (`outputs/data_metadata.json`). Return counts are the underlying sequence lengths reconstructed from the windows; they are not independent observations.

B. GAN Training

Per-epoch losses are read from the saved checkpoints by

`13_gan_training_history.py` (`outputs/gan_training_history.json`).

MLP GAN: 50 epochs, about 2 minutes, trained on all 2,296 pooled era-A rows. The discriminator loss fell from 1.21 to 0.10 while the generator loss rose from 0.68 to 6.42, which means the discriminator overpowered the generator rather than the two reaching a balance. An earlier summary file (`gan_training_summary.json`) gave different starting values (0.821 and 5.29). No script produced that file, and its reference statistics match no dataset, so it is not used.

LSTM GAN: 50 epochs, about 120 minutes. The discriminator loss stayed at 1.386–1.385 and the generator loss at 0.686–0.696 throughout, close to $2 \ln 2$ and $\ln 2$, the values for a discriminator guessing at chance; the losses show no learning. Its synthetic paths yield extreme Sharpe ratios (mean optimism about -8.8×10^5 ; median -6.07), indicating numerically unstable generation.

C. Rule Universe

500 specifications → 197 distinct names → 185 distinct rules with defined real and synthetic Sharpe ratios (180 MA crossover, 4 momentum, 1 RSI-style); 181 for the GAN generators.

D. Code Availability

Appendix E lists the code that reproduces the main results table and builds the report.

- `01_data_pipeline.py` : data collection, cleaning, windowing

- `02_baselines.py` : stationary bootstrap and GARCH generators; RQ1 fidelity
- `03_strategies.py` : rule generation
- `04_gan_generator.py` , `04b_gan_generator_lstm_FIXED.py` : MLP and LSTM GAN training
- `05_evaluation.py` , `05b_evaluation_lstm_comparison.py` : per-rule real and synthetic Sharpe ratios
- `06_bootstrap_resampling_validation.py` : earlier three-generator resampling check (superseded by `12_paired_rank_tests.py`)
- `12_paired_rank_tests.py` : paired resampling comparison of all four generators (Tables 4.4 and 4.5)
- `13_gan_training_history.py` : per-epoch GAN losses from the saved checkpoints (Appendix B)
- `14_data_description.py` : descriptive statistics and Figure 3.2 (Section 3.1)
- `10_distinct_rule_rq2.py` : RQ2 statistics on distinct rules (Section 4.2) and Figure 4.2
- `11_report_figures.py` : Figures 3.1, 4.1 and 4.3
- `07_walkforward_baseline.py` : genuine walk-forward baseline (implemented, not yet run)
- `tests/` : pytest suite (Section 3.6); `README.md` : environment, data hashes and run order
- `archive/pipeline_as_run_2026-10-05/` : the scripts exactly as they produced the reported results

E. Code Listings

E.1 Reproducing Table 4.2. The cell below reads the RQ2 statistics written by `pipeline/10_distinct_rule_rq2.py` and prints the values reported in Table 4.2. It runs when the report is built, so the printed table is regenerated from the saved outputs each time.

```
import json
import pandas as pd

r = json.load(open('outputs/rq2_distinct_rules.json'))
rows = {}
for name, g in r['generators'].items():
    lo, hi = g['fisher_ci']
    rows[name] = {'n': g['n'], 'rho': round(g['spearman_rho'], 3),
                  'CI low': round(lo, 3), 'CI high': round(hi, 3)}
pd.DataFrame(rows).T
```

	n	rho	CI low	CI high
garch	185.0	0.536	0.424	0.631
bootstrap	185.0	0.337	0.203	0.459
lstm	181.0	0.299	0.161	0.427
gan	181.0	0.189	0.045	0.326

E.2 Trade return and Sharpe ratio: as run and corrected (abridged excerpts). The reported results were produced by the first version (`archive/pipeline_as_run_2026-10-05/05_evaluation.py`); the corrected version is in `pipeline/05_evaluation.py` but has not yet been used to regenerate results (Section 3.6).

As run:

```
# moving average of returns
fast_ma = pd.Series(returns).rolling(window=ma_fast).mean()
# difference of two single-day returns
trade_return = returns[exit_idx] - returns[entry_idx]
# across the trades in one window
sharpe = np.mean(trades) / np.std(trades)
```

Corrected:

```
log_price = pd.Series(np.cumsum(returns))
# moving average of the price path
fast_ma = log_price.rolling(window=ma_fast).mean()
# hold from the day after entry to exit
position[entry_idx + 1:exit_idx + 1] = 1.0
daily = position * returns
# daily strategy returns over the window
sharpe = np.mean(daily) / np.std(daily)
```

E.3 Building the report. From `attempt-01/` :

```
.venv/bin/pip install -r requirements-lock.txt
.venv/bin/python -m playwright install chromium
# tests, contents list, execute notebook, A4 PDF + HTML, word count
./GENERATE_PDF.sh
```